UNIVERSIDAD GERARDO BARRIOS



FACULTAD DE CIENCIA Y TECNOLOGIA

TITULO DEL PROYECTO:

SISTEMA ATENEA – PROTOTIPO FUNCIONAL 1

ASIGNATURA:

PROGRAMACIÓN COMPUTACIONAL III

DOCENTE:

ING. DANIEL ALBERTO MELÉNDEZ MARTÍNEZ

ESTUDIANTES:

PEDRO PABLO TREJO GUEVARA - SMSS023925

GABRIEL ALEJANDRO RAMIREZ SEGOVIA - SMSS015825

SAMUEL ALEXANDER AGUILAR SALAMANCA - SMSS125525

FREDY ALEXANDER RIVAS LOZA - SMSS033025

OSCAR ANTONIO RAMOS CARBAJAL - SMSS036825

FECHA DE ENTREGA:

15 de noviembre de 2026

INDICE

1. INTRODUCCIÓN 3
2. OBJETIVOS 4
2.1. Objetivo General 4
2.2. Objetivos Especificos 4
3. DELIMITACIÓN DEL PROYECTO 4
4. JUSTIFICACIÓN DE LAS TECNOLOGÍAS A UTILIZAR 5
5. REQUERIMIENTOS DEL PROYECTO 5
5.1. Requerimientos Funcionales (RF) 5
5.2. Requerimientos No Funcionales (RNF) 6
6. ARQUITECTURA Y ESPECIFICACIÓN TÉCNICA DEL PROTOTIPO 6

1. INTRODUCCIÓN

En el entorno de la educación superior, los programas de tutorías académicas y refuerzos entre pares desempeñan un papel fundamental para disminuir la tasa de deserción estudiantil y mejorar el rendimiento académico. Sin embargo, en la práctica, el registro de los estudiantes beneficiados (pupilos), el control de su asistencia y el seguimiento de su progreso suelen gestionarse de manera manual o mediante hojas de cálculo aisladas. Esta problemática genera inconsistencias en los datos, pérdida de información histórica, duplicidad de registros y una total carencia de métricas visuales que permitan a los coordinadores evaluar el impacto real del programa. Además, los sistemas tradicionales a menudo descuidan aspectos críticos de seguridad en el manejo de credenciales institucionales.

Para dar solución a esta problemática, se presenta el **Sistema Atenea**, una plataforma de escritorio diseñada bajo una arquitectura modular y robusta. El sistema automatiza el almacenamiento relacional de los pupilos, protege el acceso mediante capas criptográficas avanzadas y ofrece herramientas analíticas automatizadas para la toma de decisiones, optimizando por completo la labor de los tutores y administradores académicos.

2. OBJETIVOS

2.1. Objetivo General

- Desarrollar de manera colaborativa un prototipo funcional de escritorio para el **Sistema Atenea**, que permita la gestión automatizada, segura y analítica de los registros de pupilos en un entorno académico universitario.

2.2. Objetivos Especificos

- Implementar una interfaz gráfica moderna, intuitiva y desacoplada utilizando el patrón de diseño por controladores.
- Garantizar la persistencia confiable de los datos mediante una base de datos relacional local optimizada.
- Proteger el acceso al sistema mediante un módulo de autenticación basado en funciones hash unidireccionales (SHA-256 y HMAC).
- Incorporar un módulo analítico capaz de procesar los registros y generar reportes gráficos estadísticos de rendimiento por carrera y ciclo.

3. DELIMITACIÓN DEL PROYECTO

El presente Prototipo 1 del Sistema Atenea se delimita al desarrollo e implementación de una aplicación de escritorio orientada a ejecutarse de manera local (*offline*) en equipos de los tutores y administradores del programa académico.

- **Alcance Funcional:** Comprende el inicio de sesión con validación de credenciales cifradas, el registro de nuevos pupilos (carnet, nombre, carrera y ciclo), la visualización en tiempo real de los registros almacenados en una tabla interactiva y la ejecución de scripts analíticos para generar gráficos de barras agrupados.
- **Exclusiones de la Fase 1:** Quedan excluidas de esta entrega inicial la sincronización con servidores en la nube, el despliegue web, la gestión avanzada de roles múltiples y el módulo de calendarización de sesiones de tutoría en tiempo real.

4. JUSTIFICACIÓN DE LAS TECNOLOGÍAS A UTILIZAR

La selección del stack tecnológico se ha fundamentado en criterios de rendimiento, portabilidad, seguridad y eficiencia para el trabajo colaborativo en equipo:

- **Python (Lenguaje Base):** Seleccionado por su gran versatilidad, legibilidad, tipado dinámico y su ecosistema maduro para desarrollo de software de escritorio y ciencia de datos.
- **PySide6 (Interfaz Gráfica - UI):** Permite construir interfaces gráficas profesionales, modernas y personalizables (incluyendo ventanas sin bordes nativos y barras laterales responsivas), superando las limitaciones visuales de librerías tradicionales como Tkinter.
- **SQLite3 (Base de Datos):** Base de datos relacional autocontenida y ligera. Se justifica plenamente porque no requiere la instalación de servidores externos pesados (como MySQL Server o PostgreSQL), facilitando la portabilidad del prototipo y asegurando la integridad transaccional mediante restricciones como claves primarias y campos UNIQUE.
- **Pandas y Matplotlib (Analítica):** Librerías estándar de la industria para el análisis de datos y representación gráfica. Permiten transformar consultas SQL en DataFrames limpios y exportar gráficos estadísticos claros.
- **Hashlib y HMAC / SHA-256 (Seguridad):** Librerías nativas de Python utilizadas para garantizar que las contraseñas nunca se almacenen ni viajen en texto plano, previniendo ataques de temporización (*timing attacks*) durante el inicio de sesión.

5. REQUERIMIENTOS DEL PROYECTO

5.1. Requerimientos Funcionales (RF)

- **RF-01 (Autenticación Segura):** El sistema debe permitir el acceso únicamente a usuarios autorizados mediante la validación de credenciales cifradas.
- **RF-02 (Registro de Pupilos):** El sistema debe proveer un formulario para capturar y registrar la información del estudiante (Carnet, Nombre, Carrera y Ciclo).
- **RF-03 (Control de Duplicidad):** El sistema debe validar que el carnet ingresado sea único, rechazando registros duplicados a nivel de base de datos.
- **RF-04 (Visualización Tabular):** El sistema debe mostrar en una tabla de solo lectura el listado actualizado de todos los pupilos registrados.

- **RF-05 (Módulo Analítico por Terminal):** El sistema debe permitir procesar la información almacenada para generar reportes estadísticos visuales agrupados por carrera o ciclo.

5.2. Requerimientos No Funcionales (RNF)

- **RNF-01 (Seguridad Criptográfica):** Las contraseñas deben ser procesadas utilizando algoritmos de resumen unidireccional SHA-256 junto con verificación HMAC.
- **RNF-02 (Arquitectura Modular):** El código fuente debe estar estrictamente separado en carpetas y responsabilidades (database, ui, controllers, security, analytics), garantizando un desarrollo colaborativo sin conflictos.
- **RNF-03 (Usabilidad Estética):** La interfaz gráfica debe mantener una línea visual limpia, con soporte para ventanas flotantes personalizadas y controles de usuario accesibles.
- **RNF-04 (Portabilidad):** El sistema debe poder configurarse y ejecutarse en cualquier sistema operativo compatible con Python 3 y un entorno virtual estándar.

6. ARQUITECTURA Y ESPECIFICACIÓN TÉCNICA DEL PROTOTIPO

El proyecto se encuentra organizado bajo una estructura modular profesional que facilita el mantenimiento y la escalabilidad del software:

- **main.py:** Orquestador principal que inicia la base de datos y lanza el flujo de autenticación.
- **Módulo database/:** Administra la conexión a SQLite (conexion.py) y las operaciones CRUD (crud_pupilos.py), inicializando automáticamente la tabla pupilos al arrancar.
- **Módulo ui/:** Contiene las vistas construidas con PySide6 (ventana_login.py y ventana_partenon.py) con un diseño moderno sin bordes nativos.
- **Módulo controllers/:** Separa la lógica de negocio y eventos de la interfaz mediante controlador_login.py, controlador_main.py y el validador de datos puros validations.py.
- **Módulo security/:** Implementa auth_manager.py para la validación de claves mediante hashes SHA-256 y HMAC.
- **Módulo analytics/:** Utiliza Pandas y Matplotlib en graficos_progreso.py para procesar los datos de SQLite y generar gráficos estadísticos interactivos o de exportación.